



Week 14(Memory Management)

✓ Real Memory Management(RAM)

✓ Binding of Instructions and Data to Memory

✓ Logical Address vs Physical Address

- Memory-Management Unit (MMU)
- Dynamic Linking

✓ Fixed Partition

✓ Variable Partition Multiprogramming

✓ Paging

- Paging Hardware
- Implementation of Page table
- Memory Protection
- Shared Pages
- Structured of the Page Table

✓ Segmentation

- Segmentation Architecture
- Segmentation Hardware

✓ Segmentation Paging

| **Contnet**

✓ Real Memory Management(RAM)

- if you have 4gb of ram, and your program is larger than 4gb, it can't be run
- or if you have 1gb left → more than 1 gb can't be run too

✓ Binding of Instructions and Data to Memory

Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers)
- When you load a program to memory, your program needs to be assigned memory address
- way to assign, it may
 - (static)assign at compile time, this is for low-level program in the past (.com program in window) always get loaded to same memory address, this address gets assigned at compile time → when you run the .com program, it always loads to same memory space all the time.
 - (static)load time, OS assigns memory when it loads your program → for today, you run program today, it may assign at 1000, tomorrow maybe 2000 (depend on OS). But after your program is loaded, that memory space will never be changed, your program has to be at that until finish, can't change
 - (dynamic)execution time(runtime) → can be change

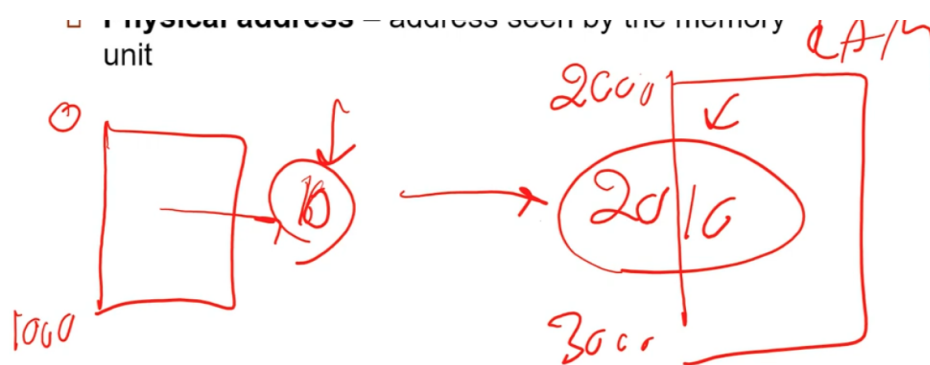
- may switch (First with 1000, switch to 2000)
- modern OS used this idea (always support dynamic binding)
 - anything that occur before run time → static

✓ Logical Address vs Physical Address

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

- Logical → address that cpu used to refer to the byte of (in) your program
- Physical → the bytes in the RAM
- they may be the same or not be the same
 - maybe the same, if cpu compile time soemthing
 - but actually, they always not be the same



- if this your program, os see your program start at 0

- but when you load your program, like it may start at 2000
 - when os say want byte 10, byte 10 is 2010
 - byte 10 → logical address

Memory Management Unit(MMU)

Memory-Management Unit (mmu)

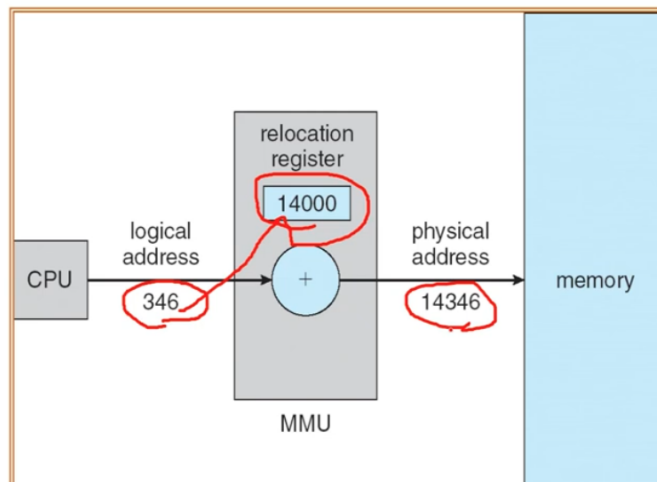
- Hardware device that maps virtual to physical address
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses

- need hardware to help mapping

Basic Idea

in mmu, has started address of your program in RAM

Dynamic relocation using a relocation register



Dynamic Linking

- OS way to try to save memory space.

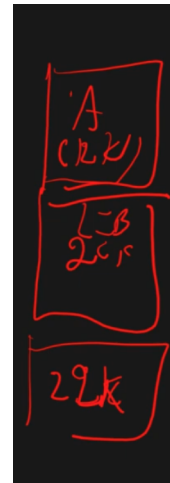
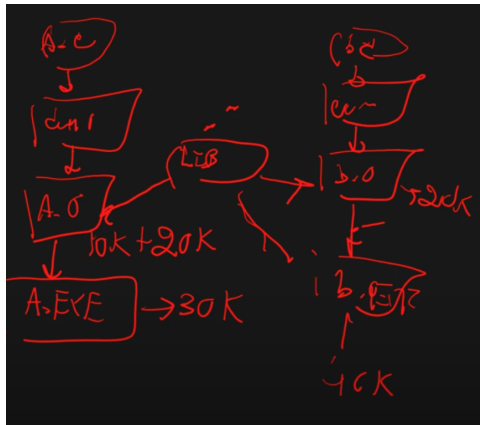
Dynamic Linking

- Linking postponed until execution time
- Small piece of code, *stub*, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system needed to check if routine is in processes' memory address
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**

How

like you have source code .c

- you compile get object code a.o
- and this a.o can't be run until you link it to some library → a.exe



- assuming this a.o has size of 10K, library is 20K $\rightarrow$ a.exe require 30k in memory (if we use static niding, linking)
- duplication of 20K
- the idea of dynamic linking is not link library directly
- like when B is loaded, it first 22 K, and when B want to use $\rightarrow$ A already load, can just use from there $\rightarrow$ save 20k

✓ Fixed Partition

a very obsolete way, we don't use anymore, maybe for the first os use

- the way os use to manage memory

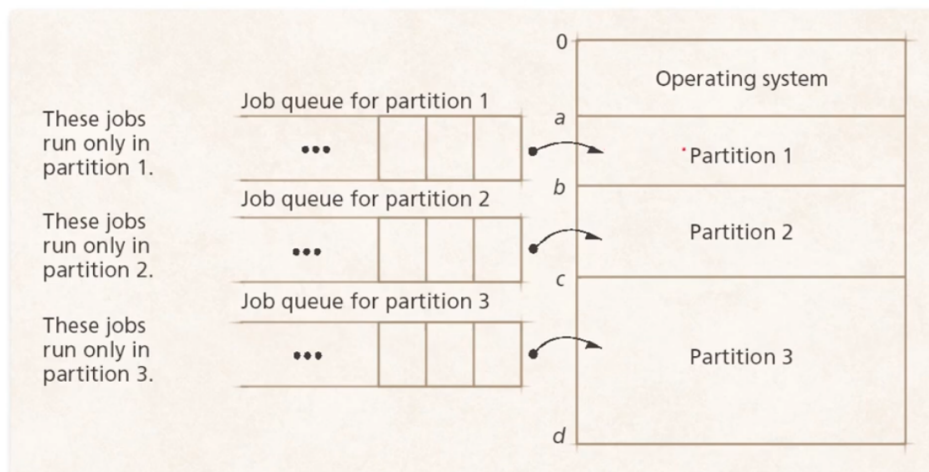
Fixed Partitions

- **Fixed Partitions:** Main memory is partitioned; one partition/job
 - Allows multiprogramming
 - Partition sizes remain static unless and until computer system id shut down, reconfigured, and restarted
 - Requires protection of the job's memory space
 - Requires matching job size with partition size

- what do we mean by fix partition
 - before the system start, the memory must be specified in advance how many partitions it gonna have
 - like system may decide memory should be partition to 4
 - in the same time, there can be 4 programs (max) running
 - size of partition has to be considered as well. (1, 2, 3, 4)
 - if program larger than 4, wait
- for fixed partitions, partitions are specified first before system start, and can't be changed

Fixed-Partition Multiprogramming

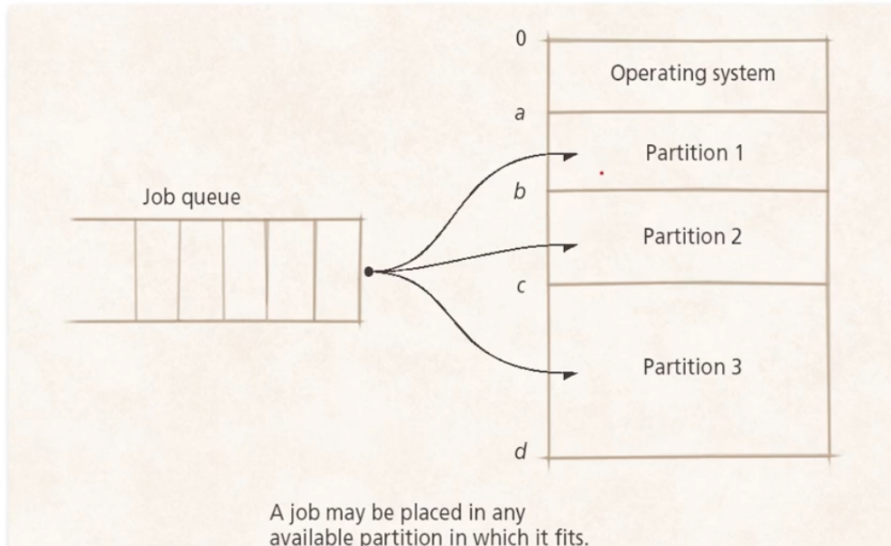
Fixed-partition multiprogramming with absolute translation and loading.



- what if we have a lot of programs that have the size of partition 1 (less than or equal to partition 1)
- larger than 1 still in partition 2 → in partition 2
- doesn't work
- like maybe partition 3 always available → this idea is better (which one available)
 - partition 1 is not available → can go to partition 2

Fixed-Partition Multiprogramming

Fixed-partition multiprogramming with relocatable translation and loading.



Fixed Partitions (continued)

To allocate memory spaces to jobs, the operating system's Memory Manager must keep a table as shown below:

Partition Size	Memory Address	Access	Partition Status
100K	200K	Job 1	Busy
25K	300K	Job 4	Busy
25K	325K		Free
50K	350K	Job 2	Busy

- OS can manage this easily, by having this table and know which is available

OS need to do protection as well

- the program in one partition should not interfere with memory in another partition

Fixed-Partition Multiprogramming

□ Protection

- Can be implemented by boundary registers, called base and limit (also called low and high)

- can only use the size of partition 2 (if it in partition2)

Idea is using

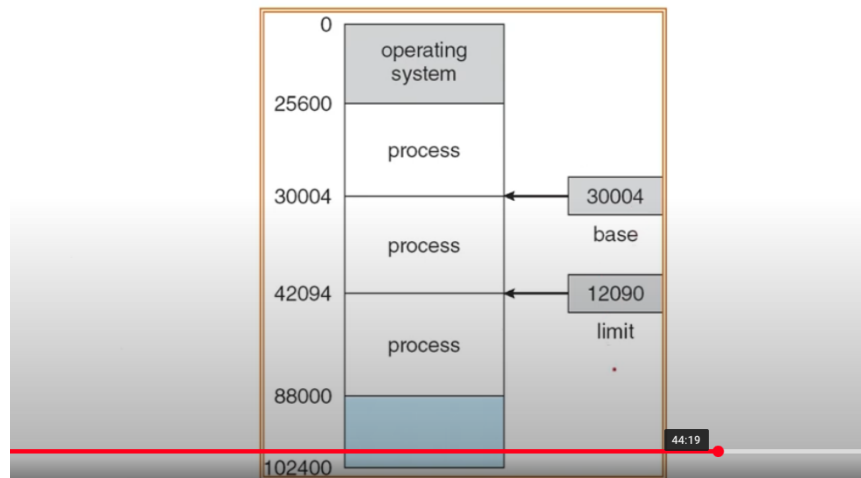
base and limit register

base → starting point of partition

limit → the size of partition

Base and Limit Registers

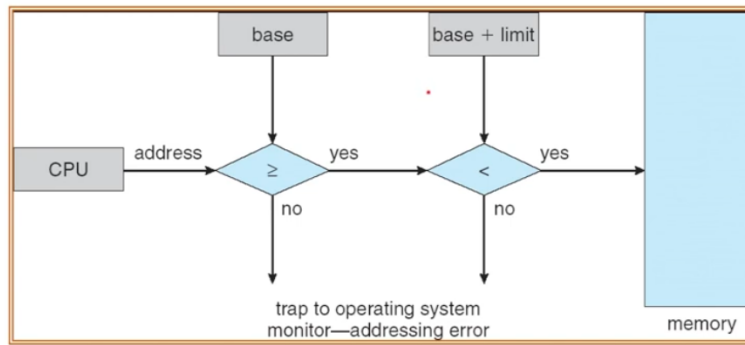
- A pair of **base** and **limit** registers define the logical address space



memory space that consider valid

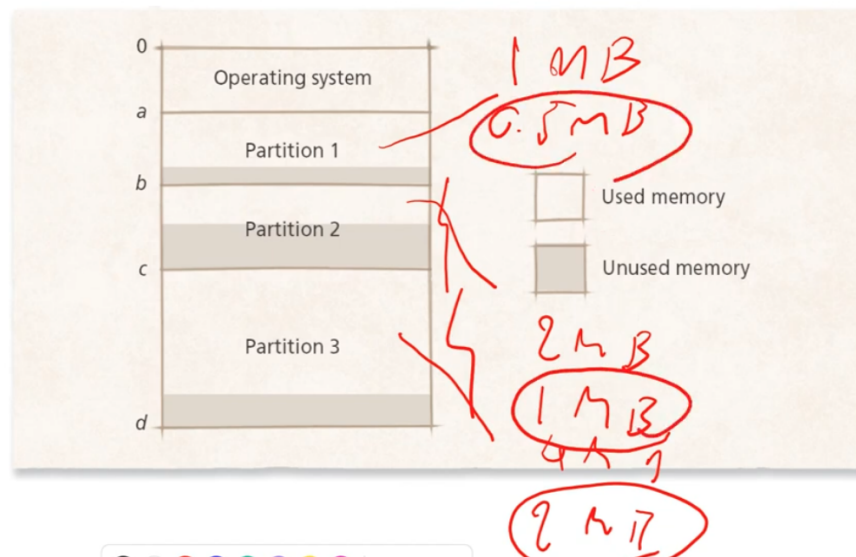
physical address

HW address protection with base and limit registers



Fixed-Partition Multiprogramming

Internal fragmentation in a fixed-partition multiprogramming system.



- internal fragmentation

- if partition 1 has 1 MB, and program running inside is 0.5 MB
- there space left, we can't use

because of fixed partition problem

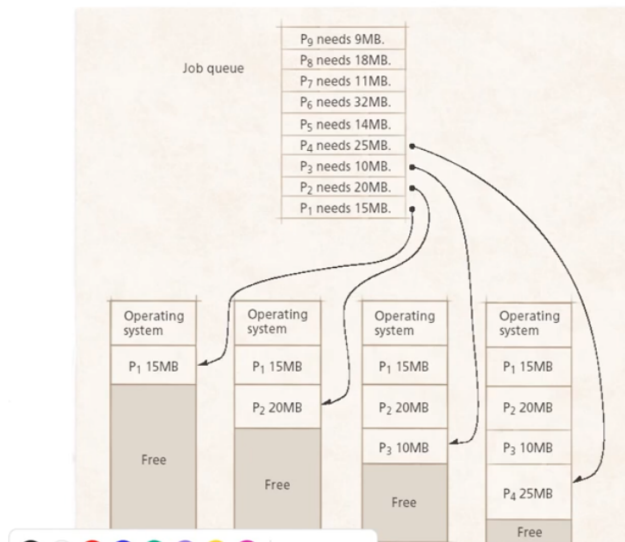
new way is introduced

✓ Variable Partition Multiprogramming

- the system doesn't define the partition in advanced

Variable-Partition Multiprogramming

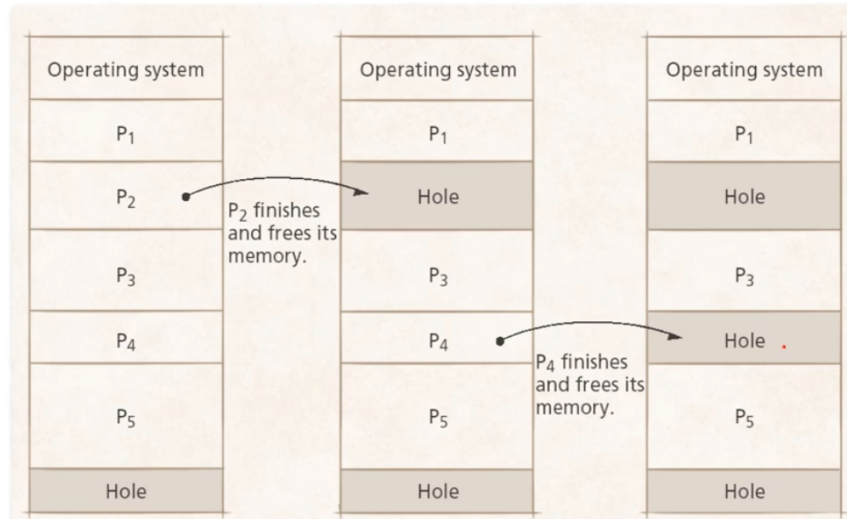
Initial partition assignments in variable-partition programming.



- it load the program and assign the space that the program need
- it can be loaded until memory is full
- this can solve some of the problem of fixed partition
 - but it may be good at the first part, but after it run → having a hole

Variable-Partition Characteristics

Memory "holes" in variable-partition multiprogramming.



- this problem is call external fragmentation

if OS has to solve →

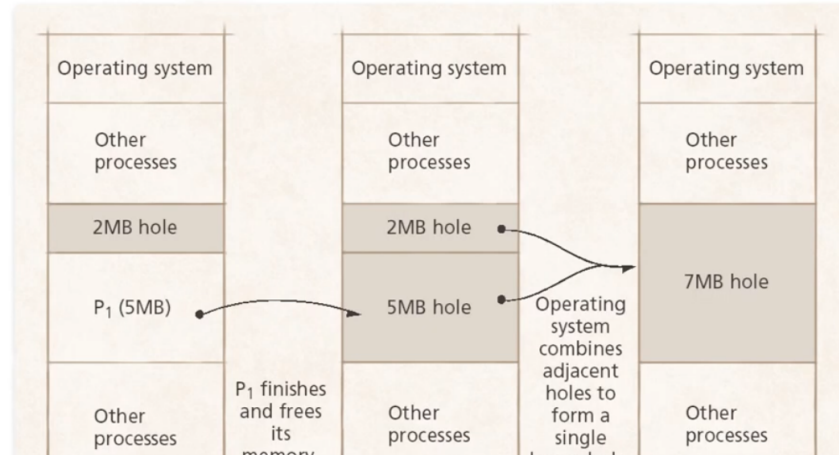
Variable-Partition Characteristics

- Several ways to combat external fragmentation
 - Coalescing
 - Combine adjacent free blocks into one large block
 - Often not enough to reclaim significant amount of memory
 - Compaction
 - Sometimes called garbage collection (not to be confused with GC in object-oriented languages)
 - Rearranges memory into a single contiguous block free space and a single contiguous block of occupied space
 - Makes all free space available
 - Significant overhead
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time

- Coalescing → if there 2 consecutive hole → combine them into 1 hole
 - can work only hole that beside to each other

Variable-Partition Characteristics

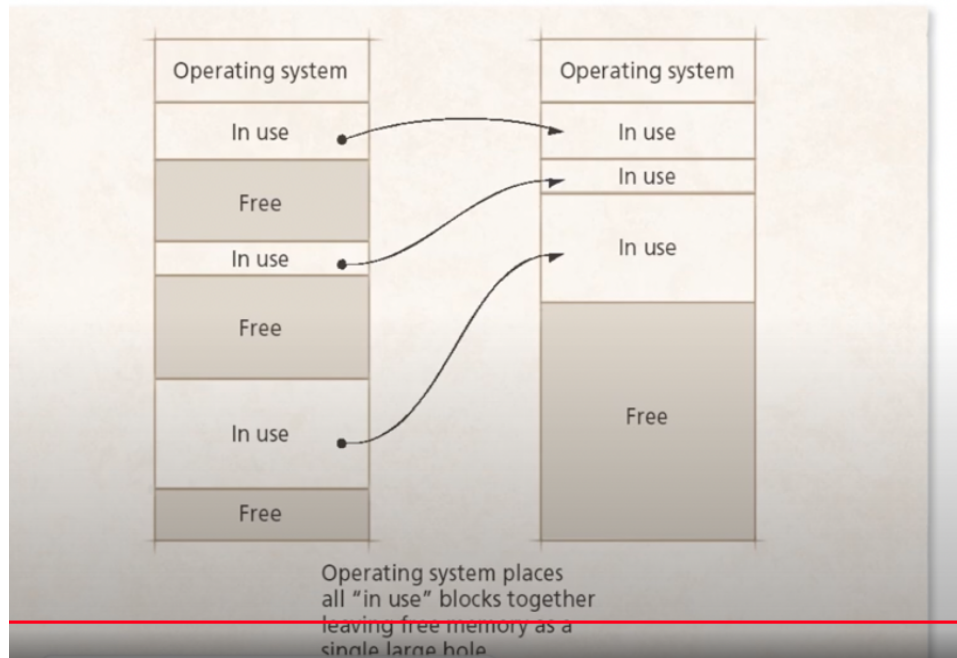
Coalescing memory "holes" in variable-partition multiprogramming.



- Compaction - it try to move process to use
 - we can only do this (dynamic binding)
 - this is not the big problem
 - when os need to rearrange the program → noone can run
 - make system slow

Variable-Partition Characteristics

Memory compaction in variable-partition multiprogramming.



There a way we try to make external fragmentation problem better, by using strategy

Memory placement strategy

Memory Placement Strategies

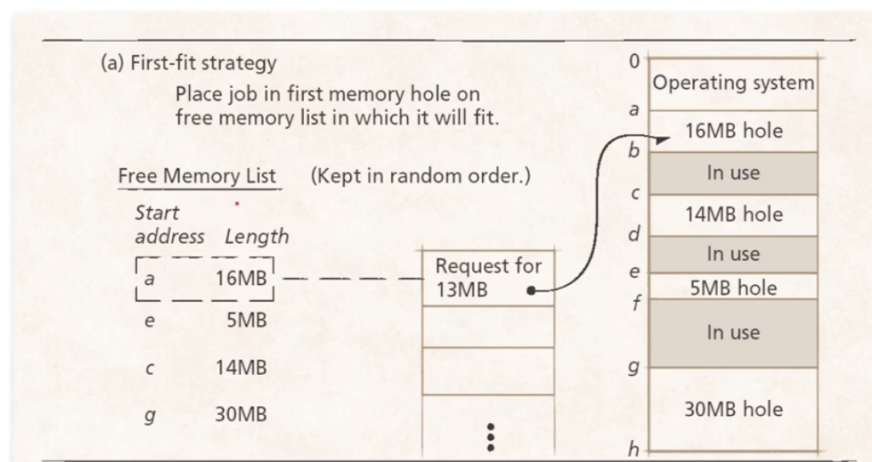
- Where to put incoming processes
 - First-fit strategy
 - ▶ Process placed in first hole of sufficient size found
 - ▶ Simple, low execution-time overhead
 - Best-fit strategy
 - ▶ Process placed in hole that leaves least unused space around it
 - ▶ More execution-time overhead
 - Worst-fit strategy
 - ▶ Process placed in hole that leaves most unused space around it
 - ▶ Leaves another large hole, making it more likely that another process can fit in the hole

1. First-fit strategy (fastest)

- start from low address
 - this one is large enough → store there

Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 1 of 3).



- 3Mb left
- next is 4 → go to 14MB hole → has 10 MB left

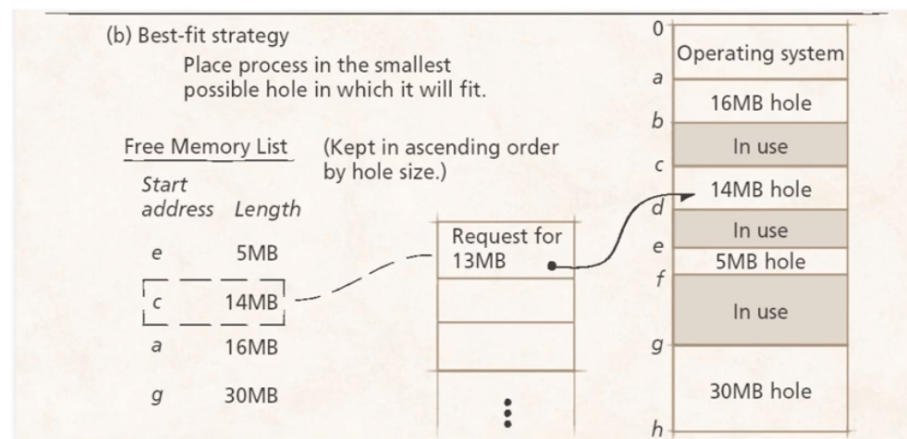
It doesn't develop to fix external fragmentation

2. Best-fit (use memory efficiently)

- find all hold and choose the one that best fit for the size of process

Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 2 of 3).

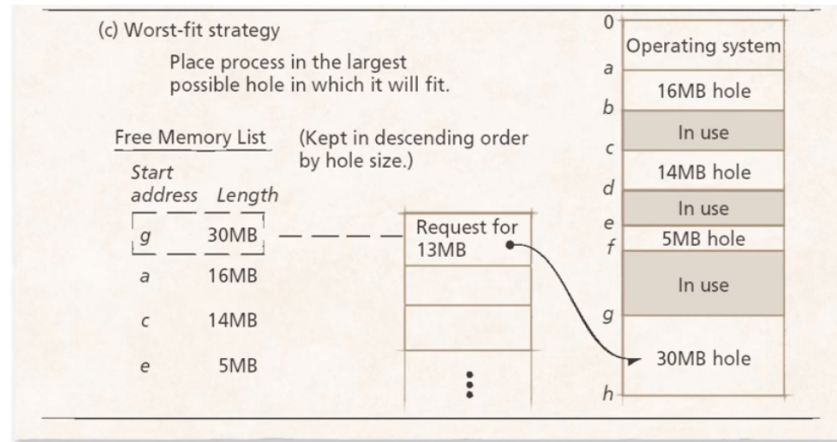


3. Worst-fit

- opposite to best fit → it use largest hole
- work slower than first fit(compare)
- but if you know that medium size → give larger hole

Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 3 of 3).



✓ Paging(Noncontiguous scheme)



the problem we faced before → contiguous scheme → load program same area

Modern computer use → Non contiguous schme

For noncontiguous → there 2 main

- paging
- segmentation

For mdoern os, we combine them adn call segmentation paging

Paging

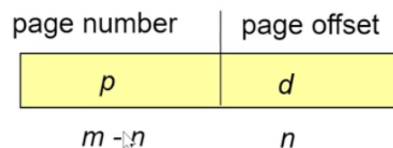
- Paging is noncontiguous memory allocation scheme
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size n pages, need to find n free frames and load program
- Set up a page table to translate logical to physical addresses
- Internal fragmentation

- Paging is one of noncontiguous memory allocation scheme
- the point is there are 2 terms
 - frame
 - page
- Idea of paging
 - the ram is divide to fix size block of power 2
 - we call each block "frame"
- in the program, we divide program into pages
 - each page has the same size as frame
- in summary → physical memory(frame), logical memory(pages)
- when your program has n pages, you have to make sure your memory has n frame
 - can be across, don't have to be consecutive to each other.
- the way to map page to frame → we use page table

- each process must have page table to map each program page to a ram frame
- paging still have internal fragmentation (very less)
- if a page has 4 bytes → if your program has the size of 35 bytes → how many pages you need → 9 → last pages used only 3 bytes (has only 1 bytes fragmentation on last page)

Address Translation Scheme

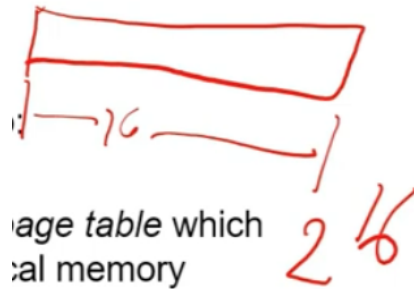
- Address (m bits) generated by CPU is divided into:
 - **Page number (p)** – used as an index into a *page table* which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit



- For given logical address space 2^m and page size 2^n

now → logical address of cpu

- it will be divide into 2 parts → page number and offset
- if we assign number to a chair, what if i divide this room using partition, and each partition has to same number of chair (like partition 1 chair 0) → now you have to refer to partition first.
- address bit?
 - if there are 16 bit → what is the maximum number of bytes(size of memory) that we can refer/used?
 - size of memory = 2 power 16



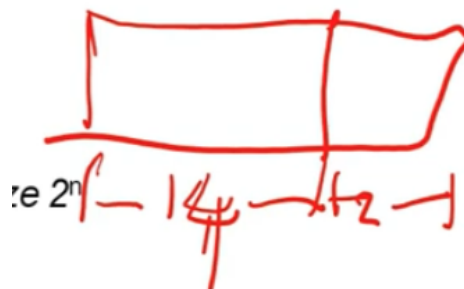
- 32 bit → 64 bit

For paging, we don't use those 16 bit as raw address

we divide address into 2 parts → just like we put partition into this room

- for paging, we divide 16 bit into 2 parts (page number and offset)
- if one page has 4 bytes → this 16 bit will be 14, 2
 - program can have upto 2 power 14 pages
 - 14 bit left for page number
 - each page has 4 bytes

memory unit
 2^{14}



memory still 2 power 16, but we change way to refer to memory address by specifying page and offset inside the page.

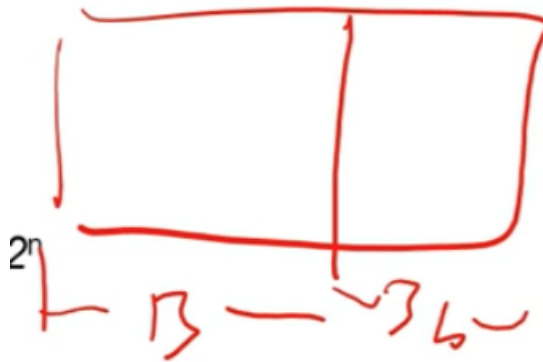
- page 0 offset 5 something

- partition number 0 chair number 5
 - bytes 5 in that page 0



2 power 16 chair → divide room that each room has 4 chair,
we have 2 power 14 rooms

What about put 8 chair to a room?



- have 2 power 13 rooms

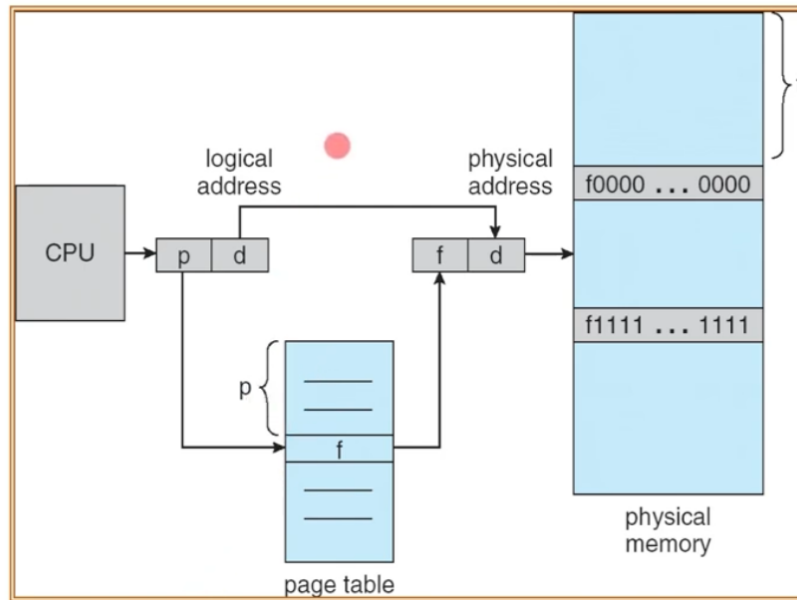
when refer to logical adress → use page number and offset.

- chair number in each room always start from 0.



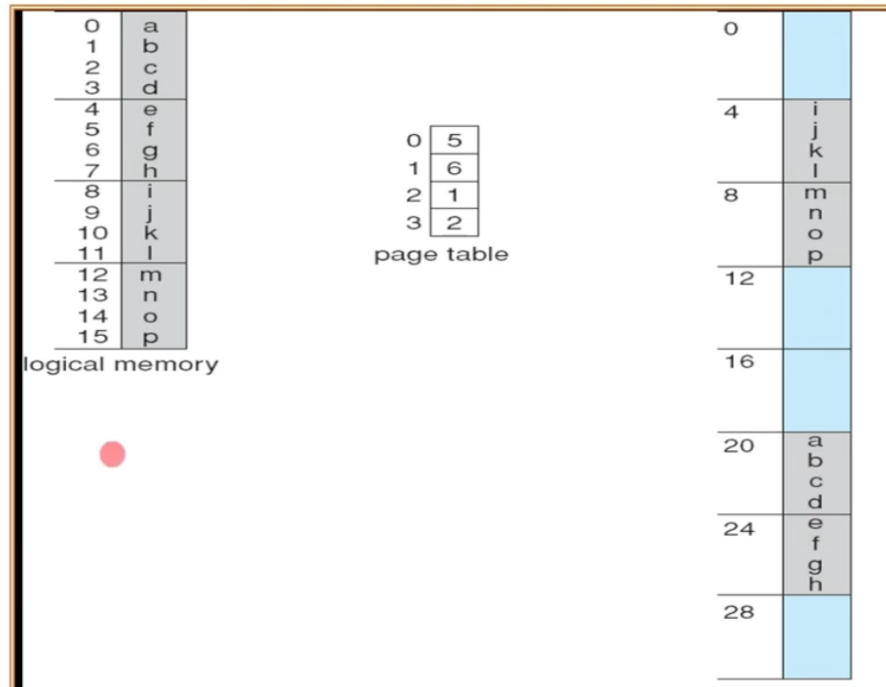
Paging Hardware(MMU)

Paging Hardware



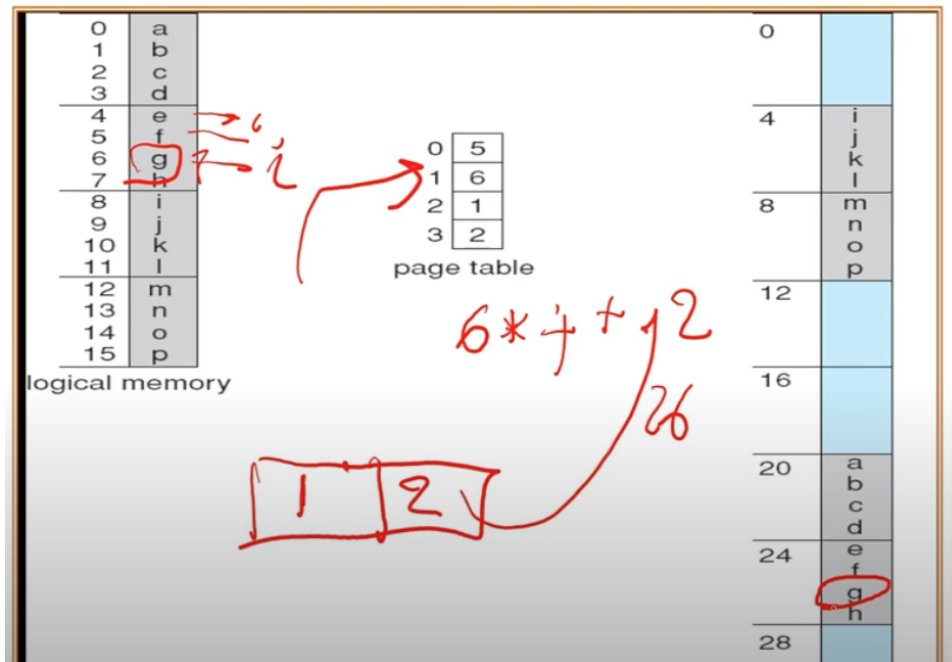
- MMU of mapping page to a frame
- CPU say that ok, i want page number p , and byte b this page
 - idea is each process ahs page table
 - p search and find that page p is loaded into frame f
 - $d \rightarrow$ offset
 - add offset

Paging Example



- we have 4 pages in this program, each pages has 4 bytes.
- this page table say that page 0 is loaded to frame 5.
- If cpu say he want g → page 1 offset 2(start count from 0)

Paging Example



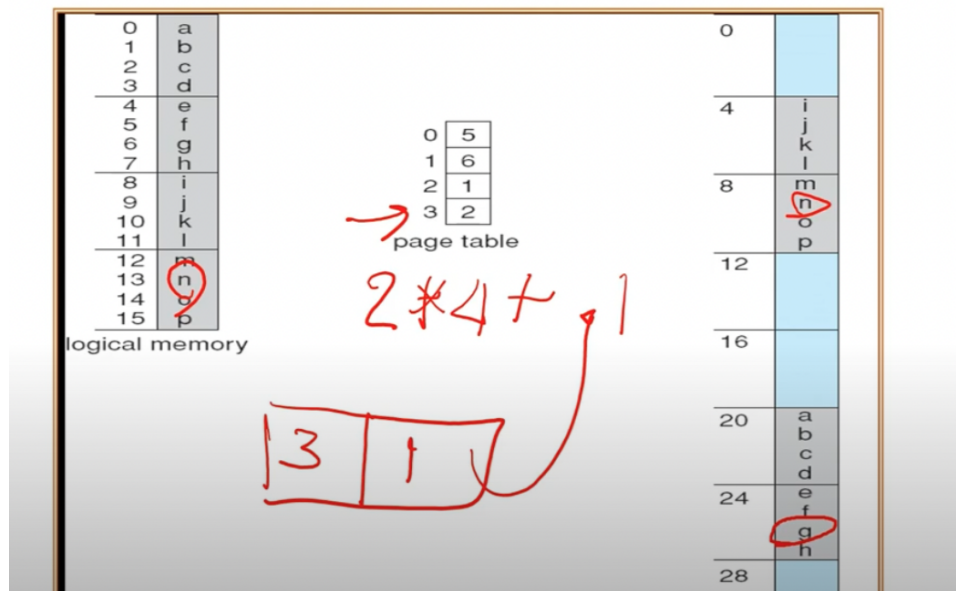
- $6 \times 4 + 2$
- if one page has 8 bytes multiply by 8

what if, prof want m

- $2 \times 4 + 0 = 8$

for n

Paging Example



Implementation of Page Table

Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

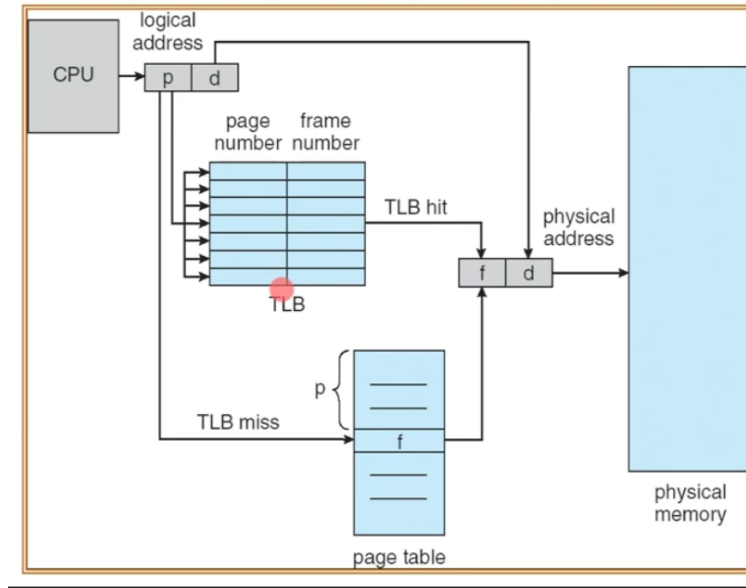
- To refer to page table, page table must be in memory

- then, we may have PTBR (register), the process when it want to access memory(page table), it must store the starting point of page table into PTBR to refer to startin gpoint of memory of page table
- the point is if your prorgam has 4 pages → it doesn't mean that page table has 4 pages
 - page table mayhave more than that
 - if we use 4 pages, this length will tell you how many pages that you have, so you don't have to go more than a page you have in program
 - like page table may have 8, but your program has 4, only go 0, 1, 2, 3 don't have to go over it.

Page table in memory,when you wna tto access data or proram in amin meory, how many time that you need to access (at least 2 times) (access page table to get a frame, then access your data)

- comparing to cpu, cpu is faster than memory
 - if we don't do anything, cpu has to wait.
- that's why we use (one way to solve problem) → TLBs.

Paging Hardware With TLB



TLB is special hardware , you can think of special memroy that is faster than RAM

- the memory that is faster than RAM, so the poit is it find information from the TLB first (this is faster thanaccess memory)
 - if it can find the frame
 - it start look for page in TLB
 - if find frame number here, then it can use this one and calculate(don't ahve to access page tbale in memory) → call TLB hit
 - cna't find → go to normal memory
- the size of TLB is smaler than page table
 - it may be update → thing from page table to TLB (if not found)

When use TLB, we use epsilon as the time you need to access TLB, and assuming that access memory require 1 microsecond

Effective Access Time

- Associative Lookup = ϵ time unit ↗ 0.1 μ s
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers 90% → 0.9
- Hit ratio = α
- **Effective Access Time (EAT)**

$$\begin{aligned} \text{EAT} &= (1 + \epsilon) \alpha + (2 + \epsilon)(1 - \alpha) \\ &= 2 + \epsilon - \alpha \end{aligned}$$

$$\text{EAT} = 2 + 0.1 - 0.9 = 1.2 \mu\text{s}$$

- and then hit ratio → the ratio that it can find the frame in TLB, then in summary, effective effective time is $2 + \epsilon - \alpha$
- we need EAT to be smallerst as possible
- the value of epsilon should be smaller, alpha should be high

what if we don't use TLB? time we need to use to get the data

- access memory 2 time → 2 microsecond

Another example

- **Effective Access Time (EAT)**

$$\begin{aligned} \text{EAT} &= (1 + \epsilon) \alpha + (2 + \epsilon)(1 - \alpha) \\ &= 2 + \epsilon - \alpha \end{aligned}$$

$$\begin{aligned} \text{EAT} &= 1.4 \mu\text{s} & 1.4 &= 2 + \epsilon - 0.8 \\ \alpha &= 80\% & 1.4 - 2 + 0.8 &= \epsilon \\ & & 0.2 & \end{aligned}$$

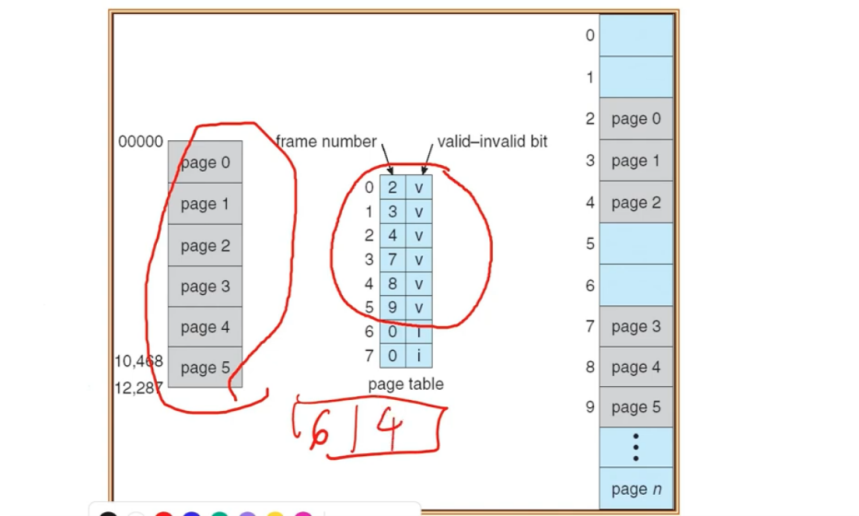
Memory Protection

- in addition to page table length register, another way to protect page table is to use valid or invalid bit in a page table

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space

Valid (v) or Invalid (i) Bit In A Page Table



- page table has 8 page site
- your program only have 6 → use part of it
- then, how to tell that the rest 2 is not valid

- if cpu said cpu want page 6 offset 4
 - system will know that page 6 is invalid → return error

each process has its own pages

Shared Pages

Shared Pages

□ Shared code

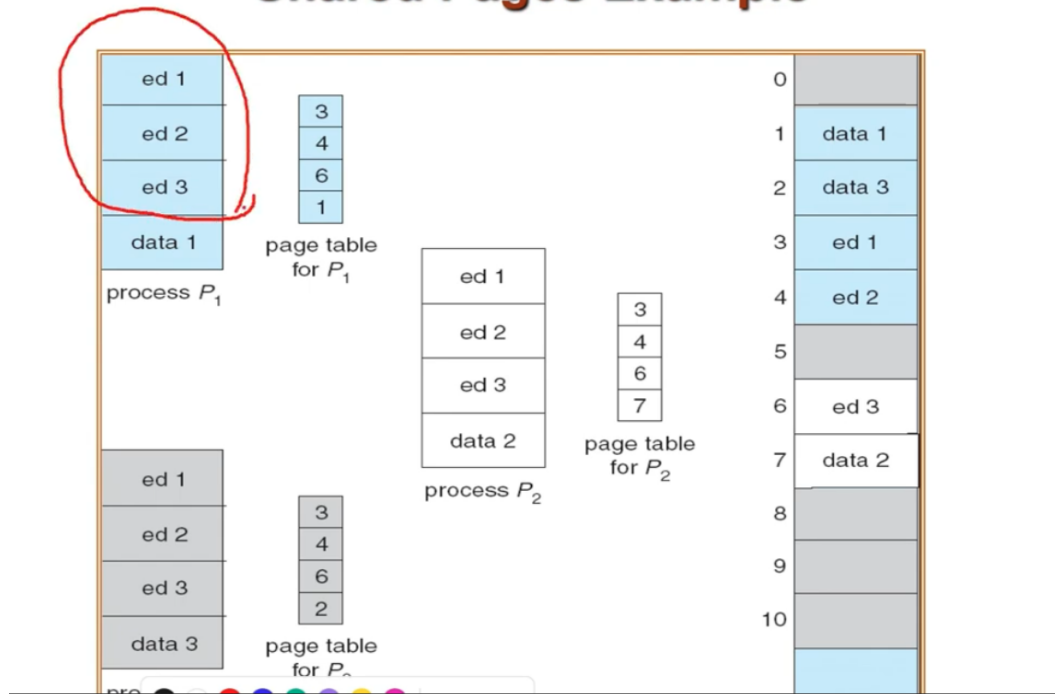
- One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
- Shared code must appear in same location in the logical address space of all processes

□ Private code and data

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space

- to save the memory, if the processes from the same program is started, the system can use the idea of share pages
- share pages is used for the pages that the data inside the page never change

Shared Pages Example



- Assuming we start 3 notepad
- assume first 3 is code
- there 1 data → the page table loaded into the same frame (3, 4, 6)
 - its a code, and the code never change
 - we can save memory (load same code to different frames → waste)

Structure of the Page Table

Structure of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

- the thing that we just learn not good if we have many pages, so there like lot of structure of pages → Hierarchical Paging
- like we may have 2 different level of pages

✓ Segmentation

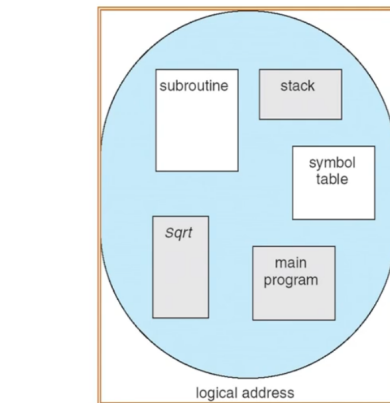
Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments. A segment is a logical unit such as:

main program,
procedure,
function,
method,
object,
local variables, global variables,
common block,
stack,
symbol table, arrays

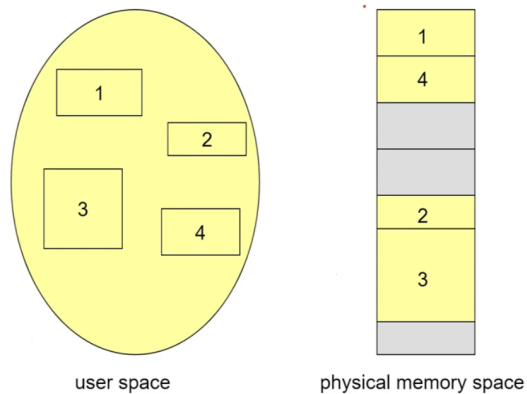
- developer never ask yourself how many pages we will used in our program
 - we now how many classes, function, but we don't think about program in term of pages

User's View of a Program



- the idea of segmentation is the program is divided into segments(main program, sqrt,...), and each segment may have different size, each can be loaded in non-contiguous memory area.

Logical View of Segmentation



- for example, you divide your program into 4 segments → you can loaded like that
 - but need to have large enough?
 - each segment may not have to same size, can be load to memory to store your segment.

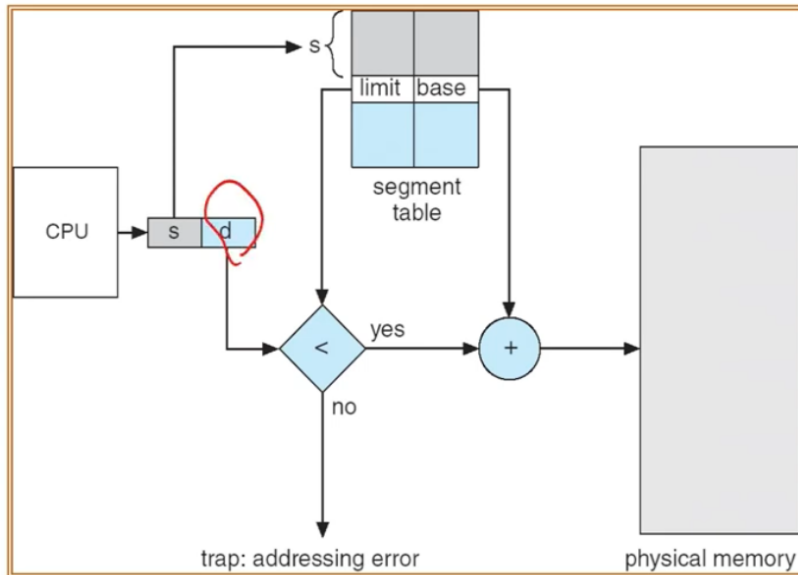
Segmentation Architecture

Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
 - **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
 - **Segment-table base register (STBR)** points to the segment table's location in memory
 - **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**
- the same idea to mapping segment logical address and physical address
 - is we need segment table
 - cpu send the segment number and bytes in that segment that it need and we need segment table to convert logical to physical

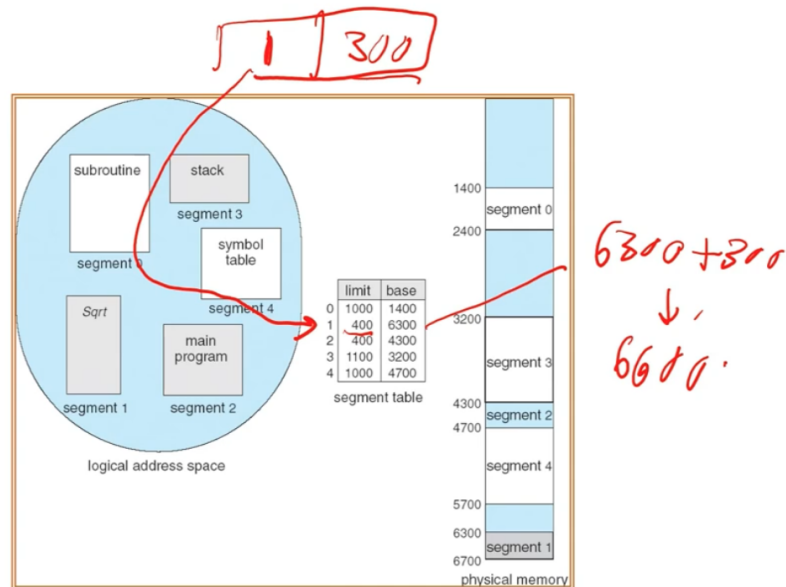
Segmentation Hardware

Segmentation Hardware

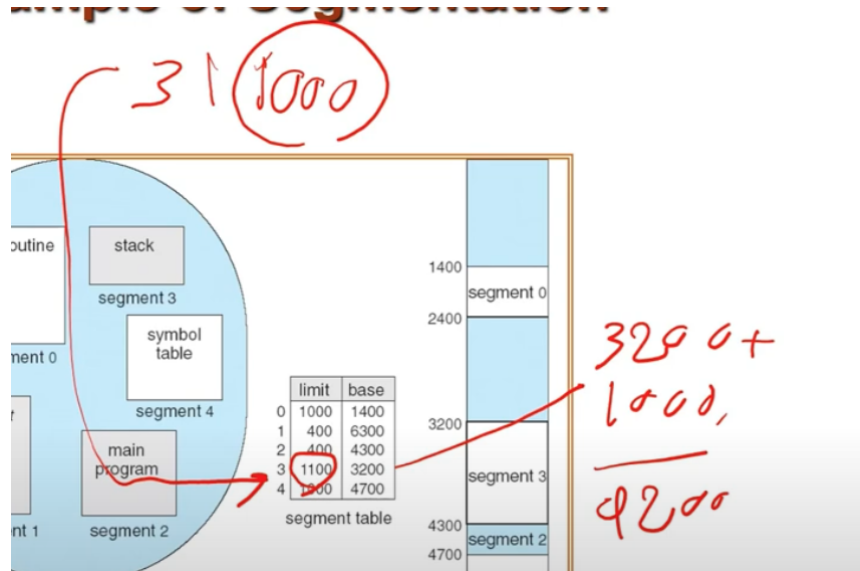


- each segmentable have limit and base
 - offset has to be less than limit to be consider valid

Example of Segmentation



- Assuming that your program has 5 segments
- cpu say want data from segment 1 offset 300
 - segment want search limit is 400 → this is valid
 - $6300 + 300 = 6600$
 - physical address = 6600



- $4 \mid 1000 \rightarrow$ invalid (not less than)

✓ Segmentation Paging

For today, we used segmentation paging

Segmentation/Paging Systems

- Process references virtual memory address

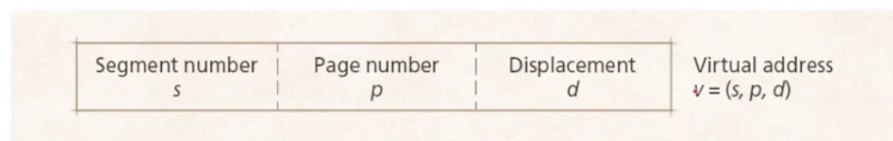
$$v = (s, p, d)$$

- DAT adds the process's segment map table base address, b , to referenced segment number, s
- $b + s$ forms the main memory address of the segment map table entry for segment s
- Segment map table entry stores the base address of the page table, s'
- Referenced page number, p , is added to s' to locate the PTE for page p , which stores page frame number p'
- System concatenates p' with displacement, d , to form real address, r

- so we think of program as segment, and inside segment is pages.
- so logical address is

Dynamic Address Translation in a Segmentation/Paging System

Virtual address format in a segmentation/paging system.



- $d \rightarrow$ offset in a page
- the page inside segment can be distributed?
- for segment, you still have to load your segment into contiguous area, what if you don't have the area that you can upload
 - when combine $\rightarrow$ you can load the page in each segment into non-contiguous area

- but you can think of program as segment

Quiz

1. Translating logical addresses to physical address is the task of

- MMU

2. One problem of fixed partitions scheme is

- internal fragmentation

3. Question

3. If a memory has four free partitions from the lower to higher addresses as follows: a = 16 GB, b = 5 GB, c = 14 GB, d = 30 GB. Assume that a new process enters the system and requests 12 GB. What partition will the process be allocated if the Best-Fit algorithm is used?

- c

4. Variable partitions scheme can cause the internal fragmentation

- False

5. The memory address accessed by the memory unit

- Physical address

6. .dll file of MS windows is an example of

- Dynamic Linking

7. The dynamic address binding is done at the ___ time

- run

8. Question

8. If a memory has 4 free partitions from the lower to higher addresses as follows: a = 16 GB, b = 5 GB, c = 14 GB, d = 30 GB. Assume that a new process enters the system and requests 12 GB. What partition will the process be allocated if the Worst-Fit algorithm is used?

- d
9. To solve the problem of variable partitions by rearranging the processes that are distributed in the memory to make a large contiguous block is called ____
- Compaction
10. To solve the problem of variable partitions by combining the adjacent free blocks into one large block is called ____.
- Coalescing
11. The memory address generated by the CPU is called ____.
- Logical Address
12. Question

12. For a fixed partition scheme, base and limit registers prevent a process from accessing the partition that belongs to others. The limit register is used to store the beginning of the partition, whereas the base register is used to store the partition size.

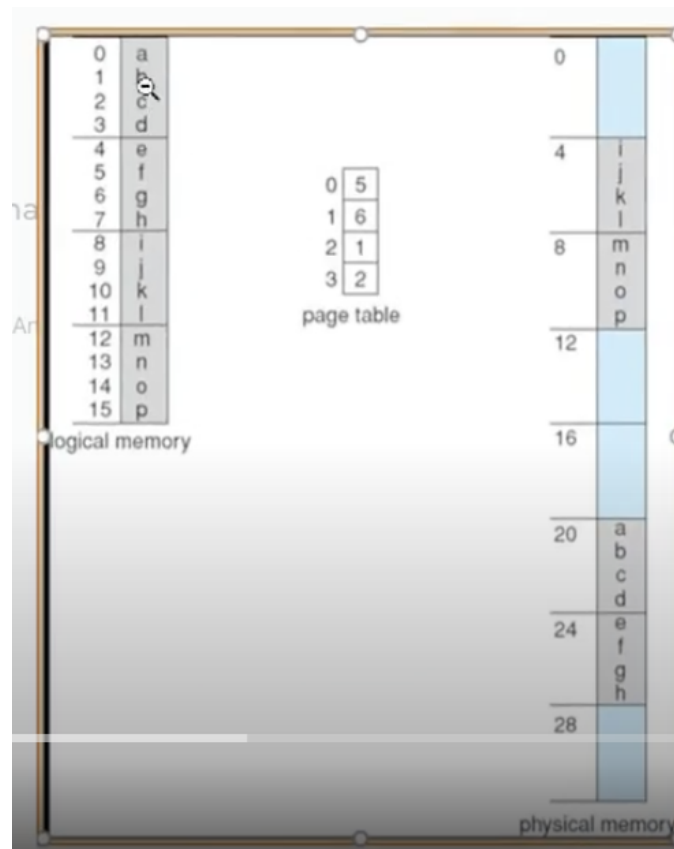
- False
13. Assuming that the base register stores 1000 and the limit stores 2000, the request to address 2100 is considered legal.
- True
14. Assuming that the base register stores 1000 and the limit stores 2000, the request to address 900 is considered legal.
- False
15. For a fixed partition scheme, the maximum number of processes that can be loaded into memory is the number of partitions
- True
16. The compaction cannot be done if address binding is done at run time
- False

17. Dividing memory into sections, each section can have a different size, but after the system is started, we cannot change the number of sections, and their size is called ____.
- Fixed Partition
18. For the dynamic linking concept, the loaded library can be shared among processes
- True
19. The main concept of dynamic linking is to load only the required parts of the program first, the libraries will be loaded when needed.
- True
20. Question

20. If a memory has four free partitions from the lower to higher addresses as follows: a = 16 GB, b = 5 GB, c = 14 GB, d = 30 GB. Assume that a new process enters the system and requests 12 GB. What partition will the process be allocated if the First-Fit algorithm is used?

- a
21. Assuming that the base register stores 1000 and the limit stores 2000, the request to address 3000 is considered legal
- False
-
1. For paging scheme, the physical memory will be divided into fixed sized blocks called page
- False (physical memory called frame) (RAM is called frame) (frame size is the same as page size)
2. For paging scheme, a program can be run if and only if numbers of free frames are greater than or equal to numbers of pages;
- True
3. If a CPU uses 16-bit address, the max memory that a CPU can address is

- 65536 bytes. (2^{16})
4. If a CPU uses 16 bit address if a page size is 16 bytes, how many bits (from 16 bits) are used for page offset?
- 4
5. If a CPU uses 16 bit address if a page size is 16 bytes, how many maximum pages in a program?
- 2^{12} (4096)
6. For paging scheme, a ___ is used for logical address to physical address translation
- Page table
7. How many pages are there in program



- 4 pages

8. first page of the program is loaded into which frame of the physical memory?

- 5
9. the logical address for "g" is page ____ and offset ____.
- 1, 2 (same as 1|2)
10. From the picture, g is stored in frame ____ and offset ____.
- 6, 2
11. Assume TLB access time is 0.25 microsecond. Memory access time is 1 microsecond. Effective Access time (EAT) is 1.35 microsecond. What is the percent of Hit ratio?

□ **Effective Access Time (EAT)**

$$EAT = (1 + \epsilon) \alpha + (2 + \epsilon)(1 - \alpha)$$

$$= 2 + \epsilon - \alpha$$

Handwritten calculations:

$$EAT = 1.4 \text{ } \mu s$$

$$\alpha = 80\%$$

$$1.4 = 2 + \epsilon - 0.8$$

$$1.4 - 2 + 0.8 = \epsilon$$

$$0.2$$

- $EAT = 2 + 0.25 - \alpha$
 - $\alpha = 2.25 - 1.35 = 0.9$
 - 90 % (0.9)
12. For paging scheme, a page table has a(n) ____ but to do the memory protection
- valid-invalid bit
13. Assume that the vi program requires 5 pages for its code and 2 pages for its data. If the shared-page concept is applied, how many frames will be shared among process of the i program?
- 5 (we share only page that never change) (5 pages can be shared)
14. The main idea of the segmentation scheme is to divide a program into parts that may not have the same size. However those parts must be

loaded into the adjacent parts of the memory.

- False (first part is true)

15. For segmentation scheme, segment table is used for logical address to physical address translation.

- True

16. In a segment table, the field that is used to specify the start address of the segment is called

- base

17. In a segment table, the field that is used to specify the size of the segment is called

- limit

18. Question

18. From the following segment table; limit base 1000 1400 400 6300 400 4300 1100 3200 1000 4700 If a logical address in the format segment-number: offset is 1:300. The physical address is _____

limit	base
1000	1400
400	6300
400	4300
1100	3200
100	4700

- offset less than limit → okay (less than 400)
- $6300 + 300$
- ans → 6600

19. Question → segment-number: offset is 2:400

- ans: error (400 not less than limit, can't equal)

20. For segmentation scheme, the external fragmentation is still exist.

- True